Opaque group-context extensions in ETK: a proof sketch

Daniel Sobral[1]

[1]*Quside Technologies, Barcelona, Spain*

(Dated: 23 October 2026, revised 25 October 2026)

MLS lets members change the extensions of the group context through a GroupContextExtensions proposal. Applications use this proposal to carry their own data, such as an authorization binding. The ETK analysis of RFC 9420 does not model the proposal. We sketch why ETK's main theorem should extend, with the same safety and injection predicates, to proposals that change only *opaque* extensions, meaning extensions that no MLS processing rule reads. In ETK's abstraction such a proposal has three visible effects: it forces a rekeying path, it updates a public field, and it appears in the committed proposal list. The first is already part of ETK's healing structure. The second carries no secret. The third is covered by ETK's consistency argument. Along the way we observe that a *constant* extension, fixed at group creation, is outside ETK's model in exactly the way every other group-context field is. This note is a sketch, not a proof. It was written against the conference version of ETK. The full version answers the first of the checks it lists. It is meant as a basis for discussion with ETK's authors.

I. THE QUESTION

Every MLS epoch carries a *group context* [1]: the protocol version, cipher suite, group identifier, epoch number, tree hash, confirmed transcript hash and a list of extensions. Applications often need to attach their own data to an epoch. RFC 9420 provides a way: a GroupContextExtensions proposal, committed like any other, replaces the extension list. In our own construction [3], an earlier design used it to write the current authorization revision into each repair commit, so that MLS would confirm the revision cryptographically.

ETK [2] proves that the core of RFC 9420, including external operations, realizes an ideal group-key functionality $\mathcal{F}_{\text{ECGKA}}$. In the safe epochs, defined by a predicate safe, application secrets are random. Its proposal actions are update, add, remove, external self-add and pre-shared key. GroupContextExtensions is not among them. A protocol that uses it therefore falls, strictly speaking, outside the theorem.

The question is whether this is a real gap or an artifact of scope. We argue that it is an artifact, for a well-delimited class of extensions.

II. WHAT THE PROPOSAL DOES

RFC 9420 fixes the following facts.

1. *Wholesale replacement* (Sec. 12.1.7). The proposal carries a list E', and the new group context has exactly E' as its extensions. Extensions not listed again disappear.

2. *At most one per commit* (Sec. 12.2). A commit with two GroupContextExtensions proposals is invalid.

3. *A path is required* (Sec. 12.4). A commit's path may be omitted only if every proposal it covers is of a type that does not require a path. Only add, psk and reinit are such types. A commit covering a GroupContextExtensions proposal therefore always carries an UpdatePath.

4. *Validity* (Sec. 12.1.7). The proposal is invalid if it sets required capabilities that some current member does not support.

5. *Where the context enters* (Secs. 6.1, 8). The group context is an input to the derivation of the joiner secret and of the epoch secret. It is also part of the content that members sign.

ETK's model is more abstract than RFC 9420 in one relevant respect. The conference text's key schedule (Fig. 2 of Ref. [2]) shows no context input. The full version (version 2.0, Appendix E) does model a group context, $\mathit{groupCtxt} = (\mathit{groupId}, \mathit{epoch}, \mathit{treeHash}, \mathit{confTransHash})$. It is an input of HKDF.Expand for the joiner secret and of HKDF.Extract for the epoch secret, and the first component of every signed content. The extension list is not part of it, and cipher suite and version are omitted too (Appendix C there).

III. SCOPE: OPAQUE EXTENSIONS

Two kinds of extension change *who may do what* in MLS. These are `external_senders`, which decides who may send external proposals, and `required_capabilities`, which decides which members are valid. Changing either affects ETK's authenticity or correctness predicates, not just public bookkeeping. We exclude both.

Definition 1 (Opaque proposal)**.** *An extension type is* opaque *if no processing rule of RFC 9420 reads extensions of that type. A GroupContextExtensions proposal with new list E', applied to old list E, is* opaque *if three conditions hold.*

First, E and E' differ only in entries of opaque types. Second, `required_capabilities` and `external_senders` are unchanged. Third, the proposal is sent by a current member.

Application-defined extensions, such as an authorization binding in a private-use code point, are opaque. The last condition matches ETK, which has no external senders.

IV. THE STATIC CASE

The simplest case needs no extension of ETK at all.

Lemma 1 (Constant extensions)**.** *Consider a group whose extension list E_0 is fixed at creation and never changed. In ETK's model E_0 does not occur, because the modeled group context omits the extension list. ETK's Theorem 1 therefore covers such a group to exactly the same extent as it covers the group identifier and every other group-context field.*

The lemma is a statement about modeling fidelity, not a reduction. It says that a constant application extension introduces no new modeling gap. This is the situation after correction R-11 in our construction, where the extension only names the authority's key and the group. ETK's full version already includes the group context as a key-derivation and signature input, without extensions. Adding the extension list to it is the refined model of Sec. V D.

V. EXTENDING ETK

A. Protocol and functionality

Let ETK^{ext} be ETK with one more proposal type.

- *Create* takes an initial list E_0.
- *Propose*(gce-E') produces a proposal carrying E'.
- *Commit* and *Process* apply E' after the other proposals. They refuse a commit with two gce proposals or with a non-opaque one. A commit covering a gce proposal always rekeys the committer's path, as RFC 9420 requires.
- *Join* reads the extension list from the group information in the Welcome.

Let $\mathcal{F}^{\text{ext}}_{\text{ECGKA}}$ be $\mathcal{F}_{\text{ECGKA}}$ with these changes:

- The proposal action set $\{\mathsf{up}, \mathsf{add}, \mathsf{rem}, \mathsf{extAdd}, \mathsf{psk}\}$ gains gce-E'.
- Each commit node c stores a public value $\mathsf{ext}(c)$, computed deterministically: $\mathsf{ext}(c) = E'$ if c commits gce-E', and $\mathsf{ext}(c) = \mathsf{ext}(\text{parent of } c)$ otherwise. Roots take E_0.
- *Process* and *Join* additionally output $\mathsf{ext}(c)$. The consistency invariant gains one clause: parties at the same node agree on ext, which holds by construction because ext is a function of the node.
- The predicates safe and inj-allowed are unchanged, *textually*.

The last point deserves a check. ETK's secrets-replaced(c, id), clause a), holds when id is the sender of c and c commits some proposal outside $\{\mathsf{add}\text{-}*, \mathsf{extAdd}\text{-}*, \mathsf{psk}\text{-}*\}$, or commits nothing. A gce proposal lies outside that set, so the clause fires for its committer. This matches the protocol, which rekeys the committer's path whenever a gce proposal is committed. The healing structure of the functionality is therefore unchanged.

B. The claim

Proposition 1 (Sketch)**.** *Under the assumptions of Theorem 1 of Ref. [2] (IND-CCA public-key encryption, SUF-CMA signatures, and key derivation in the random-oracle model), ETK^{ext} restricted to opaque proposals realizes $\mathcal{F}^{\text{ext}}_{\text{ECGKA}}$ with the predicates safe and inj-allowed of Ref. [2], Fig. 7. The advantage is the same as in that theorem.*

C. Why it should hold

The argument re-runs ETK's simulation proof with one more proposal type. It cannot be a black-box corollary of ETK's theorem. In the real protocol the gce proposal's bytes enter the confirmed transcript hash and the signed commit content, so real ETK^{ext} transcripts are not a function of real ETK transcripts. A wrapper would have to forge signatures to translate between the two. We therefore go through the four properties the proof establishes.

Secrecy. In ETK's abstraction the extension list is input to no secret computation. The new value $\mathsf{ext}(c)$ is public, chosen by the environment, and determined by the history graph, so the simulator can compute it for every node. In a gce-only commit the key derivation is that of a commit with an empty proposal list, which ETK already treats: a path is required, the committer's secrets are replaced, and nothing else changes. In a mixed commit (gce together with adds or removes), the derivation is that of the same commit without the gce proposal, which ETK treats. The hybrids for the tree secrets (ETK's selective-decryption step) and for the key schedule (random-oracle programming) see the same secret inputs as in ETK. They therefore carry over, with the same query bounds.

Healing. By Sec. 12.4 of RFC 9420, a commit covering a gce proposal rekeys its committer. The functionality's

secrets-replaced clause a) records exactly this. So corruption and healing are tracked as in ETK, and safe needs no new case.

Consistency. Two parties that process the same commit compute the same ext, because it is a deterministic function of the parent's list and the committed proposals. The committed proposals are bound by the confirmed transcript hash and the committer's signature. Disagreement on ext therefore implies disagreement on the commit content, which ETK's consistency argument already excludes, using SUF-CMA signatures and collision resistance.

Authenticity. A gce proposal is a regular member proposal. ETK's injection rule for regular proposals (inj-allowed, case b) applies unchanged: injecting one on behalf of a member requires that member's signature key *and* the parent epoch's secrets.

D. A refined model

Suppose ETK's model is refined to include the group context, as RFC 9420 does, in two places: as the context of the joiner and epoch secret derivations, and as part of every member's signed content. The argument changes in two places.

- *Random-oracle steps.* The extra input is public and computable by the simulator. Programming the oracle in a safe epoch relies only on the secret input being unknown to the adversary, so the steps still go through. The only added cost is bookkeeping for the longer query strings.
- *Signatures.* Honest parties sign longer messages. In a white-box proof the reduction obtains those signatures from the signing oracle of the SUF-CMA game like any other, so nothing changes. This is also why a black-box translation fails while a re-proof does not.

In this refined model a constant extension list (Lemma 1) is covered by the same argument, as the special case with no gce proposals.

VI. WHAT MUST BE CHECKED

This sketch was written against the conference version of ETK. The full version [2] answers the first of the following points; the others must still be verified in its Appendix J.

1. *Answered.* The protocol specification includes the group context as a public input to random-oracle calls and to signatures, without the extension list. This is the refined model of Sec. V D.
2. The proof treats commits with an empty proposal list exactly as described here, since gce-only commits reduce to them.
3. The formal consistency invariant can take the ext clause without other changes.
4. The simulator's construction of commits and group information can include public data taken from the history graph.
5. The tree-secret reduction does not depend on the shape of the proposal list beyond adds, removes and updates.

One point is settled by the functionality. $\mathcal{F}_{\text{ECGKA}}$ refuses an exposure only if some challenged epoch would then be unsafe, and it puts no restriction on corrupting randomness. For the static coalition of Ref. [3] no such refusal arises, because every epoch whose roster excludes the coalition stays safe under all its exposures. The case-by-case check is in `formal/notes/etk-full-check.md`.

VII. OUTSIDE THE OPAQUE CLASS

Three cases need more than this sketch.

- *Changing* `external_senders` changes who may inject external proposals. ETK models no external senders, so both the protocol and inj-allowed would need a new case.
- *Changing* `required_capabilities` changes which members remain valid. It affects correctness and, through member validation, possibly authenticity.
- *A GroupContextExtensions proposal sent by an external sender* needs the external-sender extension first.

VIII. QUESTIONS FOR ETK'S AUTHORS

1. Is the omission of the group context from ETK's model deliberate, and does the full protocol specification confirm it?
2. Does the proof of Theorem 1 go through, as sketched, when opaque GroupContextExtensions proposals are added as a sixth proposal action?
3. Would the authors consider external senders, which several deployments use for policy services, as a natural next extension of ETK?

[1] R. Barnes, B. Beurdouche, R. Robert, J. Millican, E. Omara, and K. Cohn-Gordon, *The Messaging Layer Security (MLS) Protocol*, RFC 9420 (IETF, 2023).

[2] C. Cremers, E. Günsay, V. Wesselkamp, and M. Zhao, ETK: External-operations TreeKEM and the security of MLS in RFC 9420, in *Eurocrypt 2026*; full version IACR ePrint 2025/229, version 2.0 (11 April 2026).

[3] D. Sobral, Two clocks, one cut: may–can concordance and revocation in MLS groups, manuscript (2026).